

```

30     unsigned int f1:4;
31     unsigned int f2:4;
32     unsigned int f3:8;
33     unsigned int f4:16;
34 } h;
35 }header_t;
36 #pragma pack()
37 static PyObject* InterfaceMethod_decode_header(PyObject *self,
PyObject *args)
38 {
39     char *buff=NULL;
40     int len=0;
41
42     header_t header;
43     PyObject *retval, *entry;
44
45     if(PyArg_ParseTuple(args,"s#",&buff, &len)==0) {
46         printf("Error in parsing header data \n");
47         PyErr_SetString(PyExc_StandardError, "Error in parsing
header data");
48         retval = Py_BuildValue("s", "FALSE");
49         return retval;
50     }
51     if(buff) {
52         header.val = (unsigned int)(strtoul(buff, NULL, 16));
53     }
54
55     printf(" f1=%x, f2=%x f3=%x f4=%x \n",
56     (unsigned int)header.h.f1,
57     (unsigned int)header.h.f2,
58     (unsigned int)header.h.f3,
59     (unsigned int)header.h.f4);
60     retval = Py_BuildValue("iiii", "TRUE", (int)header.h.f1,
(int)header.h.f2,
61                                     (int)header.h.f3,
(int)header.h.f4);
62     return retval;
63 }
64
65 static PyMethodDef InterfaceMethod[] = {
66     {"hello_world", (PyCFunction)InterfaceMethod_hello_world,
METH_NOARGS, "Prints Hello World!"},
67     {"add_integers", InterfaceMethod_add_integers, METH_VARARGS,
"Adds integers"},
68     {"decode_header", InterfaceMethod_decode_header, METH_VARARGS,
"Decodes header. TRUE on success FALSE on failure"},
69     {NULL, NULL, 0, NULL}
70 };
71
72 PyMODINIT_FUNC
73 initInterfaceMethod(void)
74 {
75     (void) Py_InitModule3("InterfaceMethod", InterfaceMethod,
"Interface Method's docstring");

```

```

76 /* COMMENT2 : Py_InitModule function without docstring */
77 /*(void) Py_InitModule("InterfaceMethod", InterfaceMethod);
*/
78 }

```

Explanation

You've got to include the key header file, *Python.h*. A peek at this file in *PYTHON_INCLUDE_PATH/python2.6/* shows that it, in turn, includes many main C header files such as *<limits.h>*, *<stdio.h>*, *<string.h>*, *<errno.h>*, *<stdlib.h>*, *<unistd.h>*, *<stddef.h>* and so on.



Note: The Python documentation recommends that you include *Python.h* at the beginning of your source code, because it defines the pre-processor directives that can affect standard headers. Variable names with a *Py* or *_py* prefix should *not* be used, as all Python APIs start with *Py*, while *_Py* is used for internal purposes.

Let's take a look at the functions. The interface functions take a pointer to the invoked object, and arguments (optional). They return a pointer to a *PyObject* class instance. *InterfaceMethod_hello_world()* just prints 'Hello World!' and returns *Py_None* (equivalent to *None* in Python). Note that the reference to the Python '*None*' object is freed if we simply return *Py_None*. *Py_INCREF* [3] ensures that the *None* object reference is not freed.

The *PyArg_ParseTuple()* [2] API function parses arguments passed to *InterfaceMethod_add_integers()* to get addends. *PyArg_ParseTuple()* returns *True* on success, while on failure it returns *False* and raises an exception. The second argument in the example, '*ijji*' specifies that two mandatory integers, followed by an optional integer, have to be parsed. The failure case sets the error message 'Error in parsing', and raises an exception of type *PyExc_StandardError* using the *PyErr_SetString* API function [5]. The '*NULL*' value is returned in case of failure. The integer sum is returned using *Py_BuildValue* [2].



Note: *NULL* or *-1* is returned as an error indicator in practice.

The *InterfaceMethod_decode_header()* is used to decode the header structure *header_t*, which has four fields: *f1*, *f2*, *f3* and *f4*, of 4, 4, 8 and 16 bits respectively. *PyArg_ParseTuple()* parses the argument into a string. The string is converted to an *unsigned long int* using *strtoul()*. *Py_BuildValue* builds Python tuples with decoded bit-field values of the header. Note that the first return value indicates *TRUE* (success) or *FALSE* (failure).



Note: You can also return a Python list using *PyList_New()* and *PyList_SetItem()* [8].